New
Search
Projects
Artifacts
Scheduled
Design
Customize
More

Pinned
HTML to JSON
Upwork Proposal
Flow AI Video Generation

Recents
Bangladeshi mango e-commerce
npm build EAGAIN spawn error
HTML email template for Eleven
Design system pages
Untitled
Image batch processing
SIM slot lock mechanism
3D animation marketplace design
Untitled design

nbyitsolution · Pro

shimanto1@s12716:~/mango-shop$ npm run build

> mango-shop@1.0.0 build
> cross-env NODE_OPTIONS="--no-deprecation --max-old-space-size=1024" next build --webpack && node scripts/postbuild.mjs

▲ Next.js 16.4.3 (webpack)
- Environments: .env

thread '<unnamed>' (1451652) panicked at /root/.cargo/registry/src/index.crates.io-1949cf8c6b5557f/tokio-1.52.1/src/runtime/scheduler/multi_thread/worker.rs:495:13:

Show more

npm build EAGAIN spawn error
19h ago

Diagnosing a thread-limit crash in Next's native build tool.

This is the same limit hitting a different part of the build. It got further than before but still failed, which confirms the problem: **the server caps how many threads your account can run, and that cap is very low.**

This time the error didn't come from `tsc`. It came from SWC, the Rust compiler Next.js uses to turn your code into JavaScript. When SWC loads, it starts one worker thread per CPU core the server reports. On a shared server that might be 32 or more, so the OS refused (`os error 11` is the same `EAGAIN` as before). The config changes reduced Next's own worker processes, but they don't control SWC's internal threads.

Try once more with thread counts forced to 1

These environment variables tell SWC (tokio and rayon), Node's I/O pool, and V8 to use the minimum number of threads:

```bash
cd ~/mango-shop
pkill -u $USER -f node

TOKIO_WORKER_THREADS=1 RAYON_NUM_THREADS=1 UV_THREADPOOL_SIZE=1 \
```

If `ps` shows many old `node` / `next` processes, kill them (`pkill -u $USER -f next`) and try again.

2. Lower the memory cap

Try a smaller heap so the build doesn't reserve more memory than the server has:

```bash
NODE_OPTIONS="--no-deprecation --max-old-space-size=2048" npx next build --webpack &&
```

Or edit the `build` script in `package.json` to use 2048 or 3072 instead of 4096.

3. Skip the type-check subprocess (fastest workaround)

In `next.config.ts`:

```ts
const nextConfig = {
  typescript: { ignoreBuildErrors: true },
  // ...your existing config
};
export default nextConfig;
```

npm build EAGAIN spawn error
19h ago

Then run types separately, which uses far less memory:

```bash
npx tsc --noEmit
```

Remove `ignoreBuildErrors` once the server problem is fixed, so type errors can fail the build again.

4. Raise the process limit (if `ulimit -u` is low)

On a VPS you control, add to `/etc/security/limits.conf`:

```
shimanto1  soft  nproc  4096
shimanto1  hard  nproc  8192
```

Log out and back in, then check with `ulimit -u`. On shared hosting, ask the provider to raise it.

5. Add swap if RAM is small

```bash
sudo fallocate -l 4G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
```

6. Try an LTS Node version

Node v26 is very new. Switching to an LTS release can rule out Node-specific problems:

```bash
nvm install --lts && nvm use --lts
```

npm build EAGAIN spawn error
19h ago

Start with steps 1 and 3. Step 3 gets the build working right away, and step 1 shows you whether the real cause is process limits or memory. If you paste the output of `ulimit -u`, `ps -u $USER | wc -l`, and `free -m`, I can tell you which fix applies.

towfi@Towfique UCRT64 ~
$ ssh -p 22 shimanto1@69.57.172.207
shimanto1@69.57.172.207's password:
shimanto1@s12716:~$ npx tsc --noEmit
Need to install the following packages:
tsc@2.0.4
Ok to proceed? (y) y
npm warn deprecated tsc@2.0.4: Package no longer supported. Contact Support at https://www.npmjs.com/support for more info.

Show more

Weighing worker limits against building locally and uploading ▸

Your server is a shared "website container": no `sudo`, so swap and system limits are off the table. The `EAGAIN` means the container won't let Node start another process, either because of a process-count cap or a memory cap. Switching Node versions didn't change that. You have two options: make the build use fewer processes (steps 1–4), or build on your own PC and upload the result (step 5), which always works.

(About `npx tsc`: you ran it in your home folder, so npx downloaded an unrelated package called `tsc`. Run it inside `~/mango-shop`, where the real TypeScript is installed.)

Step 1: See what limit you're hitting

```bash
cd ~/mango-shop
ulimit -u
ps -u $USER | wc -l
cat /sys/fs/cgroup/pids.max /sys/fs/cgroup/pids.current 2>/dev/null
cat /sys/fs/cgroup/memory.max 2>/dev/null
```

npm build EAGAIN spawn error
19h ago

If `pids.current` is close to `pids.max`, or the `ps` count is close to `ulimit -u`, the process limit is the problem. A small `memory.max` (for example 1–2 GB) means memory is the problem.

Step 2: Kill leftover Node processes

Failed builds can leave processes running that use up your quota:

```bash
pkill -u $USER -f next
pkill -u $USER -f node
ps -u $USER | wc -l
```

The count should drop. Then try `npm run build` again. If it still fails, go to step 3.

Step 3: Make Next.js build with one worker

```bash
cat next.config.ts
nano next.config.ts
```

Add these keys inside your existing config object, keeping everything else:

```ts
const nextConfig = {
  // ...your existing settings
  typescript: { ignoreBuildErrors: true },
  experimental: {
    cpus: 1,
    workerThreads: false,
    webpackBuildWorker: false,
  },
};
```

npm build EAGAIN spawn error
19h ago

Save with `Ctrl+O`, Enter, then exit with `Ctrl+X`.

Step 4: Lower the memory cap and rebuild

```bash
nano package.json
```

In the `build` script, change `--max-old-space-size=2048` to `1536`. If `memory.max` in step 1 showed less than 2 GB, use `1024`. Then:

```bash
```

```
pkill -u $USER -f node
npm run build
```

If the build gets past "Creating an optimized production build", it's working. If it still fails with `spawn ... EAGAIN`, the container can't spare even one extra process, so use step 5.

Step 5 (reliable fix): Build on your PC, upload to the server

This works on any host because the server only has to run the site, not build it.

On your Windows PC, in a copy of the project with the same `.env` file:

bash

```
npm ci
npm run build
```

Copy the `.env` too, because any `NEXT_PUBLIC_` values get baked in at build time.

Upload the build output from your PC's project folder:

npm build EAGAIN spawn error
19h ago

bash

```
scp -P 22 -r .next shimanto1@69.57.172.207:~/mango-shop/
```

If `scripts/postbuild.mjs` writes files anywhere other than `.next`, such as `public/`, upload those too.

On the server, install only the runtime dependencies and start the site:

bash

```
cd ~/mango-shop
npm ci --omit=dev
npm start
```

Use whatever start method your host uses, such as PM2 or the panel's Node app setting.

Repeat step 5 each time you deploy changes. If this gets tedious, a GitHub Actions workflow can build and upload automatically on every push.

If you run step 1 and paste the output, I can tell you whether steps 3–4 can work on this server or whether you should go straight to step 5.

Add these keys inside your existing config object, keeping everything else: ts const…

Should I place like this?

Yes, but **merge** these keys into your existing config rather than pasting a second `const nextConfig` block. Your file should still have only one `nextConfig` and one `export default`.

npm build EAGAIN spawn error
19h ago

For example, if your current file looks like this:

ts

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  images: {
    remotePatterns: [{ protocol: "https", hostname: "example.com" }],
  },
};

export default nextConfig;
```

it should become:

ts

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  images: {
    remotePatterns: [{ protocol: "https", hostname: "example.com" }],
  },
  typescript: { ignoreBuildErrors: true },
  experimental: {
    cpus: 1,
    workerThreads: false,
    webpackBuildWorker: false,
  },
};

export default nextConfig;
```

npm build EAGAIN spawn error
Watch out for two things:

19h ago

Watch out for two things:

- If you already have a `typescript:` or `experimental:` section, add the new lines inside it instead of creating a second one. A duplicate key silently overrides the first.
- Keep the commas between sections, or the build fails with a syntax error.

After saving, run:

```bash
pkill -u $USER -f node
npm run build
```

If you're unsure, run `cat next.config.ts`, paste the output here, and I'll send back the exact file to use.

shimanto1@s12716:~/mango-shop$ npm run build

> mango-shop@1.0.0 build
> cross-env NODE_OPTIONS="--no-deprecation --max-old-space-size=1024" next build --webpack && node scripts/postbuild.mjs

▲ Next.js 16.4.3 (webpack)
- Environments: .env

thread '<unnamed>' (1451652) panicked at /root/.cargo/registry/src/index.crates.io-1949cf8c6b5b557f/tokio-1.52.1/src/runtime/scheduler/multi_thread/worker.rs:495:13:

Show more

Diagnosing a thread-limit crash in Next's native build tool.

This is the same limit hitting a different part of the build. It got further than before but still failed, which confirms the problem: **the server caps how many threads your account can run, and that cap is very low.**

npm build EAGAIN spawn error
19h ago

This time the error didn't come from `tsc`. It came from **SWC**, the Rust compiler Next.js uses to turn your code into JavaScript. When SWC loads, it starts one worker thread per CPU core the server reports. On a shared server that might be 32 or more, so the OS refused (`os error 11` is the same `EAGAIN` as before). The config changes reduced Next's own worker processes, but they don't control SWC's internal threads.

Try once more with thread counts forced to 1

These environment variables tell SWC (tokio and rayon), Node's I/O pool, and V8 to use the minimum number of threads:

```bash
cd ~/mango-shop
pkill -u $USER -f node

TOKIO_WORKER_THREADS=1 RAYON_NUM_THREADS=1 UV_THREADPOOL_SIZE=1 \
NODE_OPTIONS="--no-deprecation --max-old-space-size=1024 --v8-pool-size=1" \
npx next build --webpack && node scripts/postbuild.mjs
```

If Node complains about `--v8-pool-size`, remove just that flag and run the command again.

Paste this too, so we can see how tight the limits are:

```bash
nproc
ulimit -u
cat /sys/fs/cgroup/pids.max 2>/dev/null
```

If it still crashes, build on your PC

If you get the same `can't spawn worker thread` panic, this host can't run a Next.js build at all, and no config change will fix that. Use step 5 from earlier: run `npm run build` on your Windows PC, upload the `.next` folder with `scp`, then run `npm ci --omit=dev` and `npm start` on the server. Running a built Next.js site needs far fewer threads than building it, so the server should handle that part.

npm build EAGAIN spawn error
19h ago

If the build works with the variables above, I can show you how to save them in your `package.json` build script so `npm run build` uses them every time.

If it still crashes, build on your PC

I have your provided 'mango-shop.zip' guide step by step

Weighing config-loading risks for packaging the server build ▸

Here's the full process: build on your Windows PC, then upload only the finished site to the server. Use the same **UCRT64 terminal** you SSH from, since it has `tar` and `scp`.

Step 1: Install Node.js on your PC

Download the LTS installer (v24) from https://nodejs.org and install it with the default options. Then close and reopen UCRT64 and check:

```bash
node -v
npm -v
```

If UCRT64 says `node: command not found`, run this once and check again:

```bash
export PATH="$PATH:/c/Program Files/nodejs"
```

npm build EAGAIN spawn error
19h ago

Step 2: Extract the zip

Right-click `mango-shop.zip`, choose **Extract All**, and extract it to `C:\projects\`. You should end up with `C:\projects\mango-shop\package.json`. Then:

```bash
cd /c/projects/mango-shop
ls
```

You should see `package.json`, `next.config.ts`, `src` or `app`, and so on. If you see another `mango-shop` folder inside, `cd` into it.

Step 3: Copy the `.env` from the server

The build needs the same environment values the live site uses:

```bash
scp -P 22 shimanto1@69.57.172.207:~/mango-shop/.env .
```

Step 4: Install and build

```bash
npm install
npm run build
```

This takes a few minutes. When it finishes, you should see a list of routes and no errors. If it fails with type or code errors, paste them here.

Then check whether the project uses standalone output:

```bash
ls .next/standalone
```

Note whether the folder exists, because it decides step 5.

npm build EAGAIN spawn error
19h ago

Step 5: Pack the build into one file

A) If `.next/standalone` exists, it's a self-contained server:

```bash
tar -czf build.tar.gz --exclude=.next/cache .next public package.json
```

B) If it doesn't exist:

```bash
tar -czf build.tar.gz --exclude=.next/cache .next public package.json package-lock.jso
```

Step 6: Upload it

```bash
scp -P 22 build.tar.gz shimanto1@69.57.172.207:~/mango-shop/
```

Step 7: Unpack on the server

```bash
ssh -p 22 shimanto1@69.57.172.207
cd ~/mango-shop
pkill -u $USER -f node
rm -rf .next
tar -xzf build.tar.gz
```

Since you build on your PC now, also undo the `experimental` and `ignoreBuildErrors` edits to the server's `next.config.ts`. Option B overwrites that file with the zip's version anyway.

Step 8: Start the site

A) Standalone:

npm build EAGAIN spawn error
19h ago

```bash
cd ~/mango-shop/.next/standalone
cp ../../.env .
PORT=3000 node server.js
```

B) Normal:

```bash
cd ~/mango-shop
npm ci --omit=dev
npm start
```

Open the site in your browser. If it loads, stop the server with `Ctrl+C` and start it the way your host expects, such as PM2 or the Node.js app setting in your hosting panel, with the same command.

If startup crashes with the same `can't spawn worker thread` error, start it with the low-thread settings:

```bash
TOKIO_WORKER_THREADS=1 RAYON_NUM_THREADS=1 UV_THREADPOOL_SIZE=1 npm start
```

For future updates, change the code on your PC and repeat steps 4–8. Step 3 is only needed if `.env` changes.

npm build EAGAIN spawn error
19h ago

towfi@Towfique UCRT64 ~/Documents/mango-shop/mango-shop
$ npm run build

> mango-shop@1.0.0 build
> cross-env NODE_OPTIONS="--no-deprecation --max-old-space-size=4096" next build && node scripts/postbuild.mjs

▲ Next.js 16.4.3 (Turbopack)
- Environments: .env
✓ Running next.config.ts took 51ms
- Experiments (use with caution):

Show more

Working out how to rebuild native modules for Linux deployment ▸

The build worked. The last line shows your `postbuild` script already copied `public/` and `.next/static` into `.next/standalone`, so that one folder is the complete site. You only need to upload it.

Step 1: Pack only the standalone folder (on your PC)

```bash
cd ~/Documents/mango-shop/mango-shop
tar -czf build.tar.gz -C .next standalone
ls -lh build.tar.gz
```

Step 2: Upload it

```bash
scp -P 22 build.tar.gz shimanto1@69.57.172.207:~/mango-shop/
```

Step 3: Unpack on the server

```bash
ssh -p 22 shimanto1@69.57.172.207
cd ~/mango-shop
pkill -u $USER -f node
rm -rf .next/standalone
mkdir -p .next
tar -xzf build.tar.gz -C .next
cp .env .next/standalone/
```

npm build EAGAIN spawn error
19h ago

Step 4: Start it

```bash
cd ~/mango-shop/.next/standalone
TOKIO_WORKER_THREADS=1 RAYON_NUM_THREADS=1 UV_THREADPOOL_SIZE=1 \
PORT=3000 HOSTNAME=0.0.0.0 node server.js
```

You should see `✓ Ready`. Use the port your hosting panel assigned to this site if it isn't 3000. Open the site, check a product page and `/admin`, then stop it with `Ctrl+C` and set your panel's or PM2's start command to the same thing: run `node server.js` from `~/mango-shop/.next/standalone`.

If it fails with a "module" or "sharp" error

You built on Windows, so packages with native code, like `sharp` for image resizing, were installed in their Windows versions. If startup or image loading fails with an error mentioning `sharp`, `win32`, or `invalid ELF header`, run this on the server:

```bash
cd ~/mango-shop/.next/standalone
npm install --no-save --os=linux --cpu=x64 sharp
```

Then start it again. If a different package name appears in the error, paste it here and I'll give you

the fix.

For future updates, change the code on your PC, run `npm run build`, and repeat steps 1–4.

npm build EAGAIN spawn error
19h ago

Pasted text.txt

TXT

cd

Diagnosing a Turbopack module-naming bug.

The server now **starts** (`✓ Ready`), so the thread problem is solved. The new errors come from building on Windows with **Turbopack**.

What's wrong

Turbopack doesn't bundle some server packages like `pino`, `pg`, and `@aws-sdk/client-s3`. Instead it creates links with hashed names (`pino-28069d5257187539` → `node_modules/pino`) inside `.next/node_modules`. On Windows these links are Windows-style and don't survive the trip to Linux, so the server can't find those modules.

Your server's build script used `--webpack`, but the one on your PC doesn't (your PC log says "Turbopack"). Webpack doesn't use these hashed links, so rebuilding with it fixes this.

Step 1: Rebuild with webpack (on your PC)

```bash
cd ~/Documents/mango-shop/mango-shop
rm -rf .next
npx cross-env NODE_OPTIONS="--no-deprecation --max-old-space-size=4096" next build --w
node scripts/postbuild.mjs
```

The first line of output should now say `(webpack)`, not `(Turbopack)`.

Check that the packages from the errors were included:

```bash
ls .next/standalone/node_modules | grep -E "^pino$|^pg$"
ls .next/standalone/node_modules/@aws-sdk | grep client-s3
```

npm build EAGAIN spawn error
19h ago

You should see `pino`, `pg`, and `client-s3`.

Step 2: Pack and upload (on your PC)

```bash
rm -f build.tar.gz
tar -czf build.tar.gz -C .next standalone
scp -P 22 build.tar.gz shimanto1@69.57.172.207:~/mango-shop/
```

Step 3: Replace and start (on the server)

```bash
cd ~/mango-shop
pkill -u $USER -f node
rm -rf .next/standalone
tar -xzf build.tar.gz -C .next
cp .env .next/standalone/
cd .next/standalone
TOKIO_WORKER_THREADS=1 RAYON_NUM_THREADS=1 UV_THREADPOOL_SIZE=1 \
PORT=3000 HOSTNAME=0.0.0.0 node server.js
```

Open the site and check the homepage, a product page, and `/admin`.

Make webpack the default (optional)

To avoid this next time, edit `package.json` on your PC and add `--webpack` to the build script:

```json
"build": "cross-env NODE_OPTIONS=\"--no-deprecation --max-old-space-size=4096\" next b
```

After that, plain `npm run build` works for every deploy.

npm build EAGAIN spawn error
19h ago

If any error remains after this, paste just the **first 10–15 lines.** The error repeats for every request, so one copy is enough.

You should see pino, pg, and client-s3.

There's only Client-s3

There's only client-s3

towfi@Towfique UCRT64 ~/Documents/mango-shop/mango-shop
$ node scripts/postbuild.mjs
postbuild: copied public/ and .next/static into .next/standalone

towfi@Towfique UCRT64 ~/Documents/mango-shop/mango-shop
$ ls .next/standalone/node_modules | grep -E "^pino$|^pg$"

towfi@Towfique UCRT64 ~/Documents/mango-shop/mango-shop
$ ls .next/standalone/node_modules/@aws-sdk | grep client-s3
client-s3/

towfi@Towfique UCRT64 ~/Documents/mango-shop/mango-shop
$

Working out which packages need bundling versus installing.

That may be fine. With webpack, `pino` and `pg` are often **bundled into the server code** instead of being loaded from `node_modules`, so they don't need their own folders. Check whether the build still expects them as separate packages:

```bash
grep -rlE "require\(\"(pino|pg)\"\)|from \"(pino|pg)\"" .next/standalone/.next/server
```

npm build EAGAIN spawn error
19h ago

If that prints nothing

They're bundled. Continue with **step 2** from my last message: pack, upload, and start.

If it prints file names

The build still loads them from `node_modules`, so add them yourself. First find the exact versions your project uses:

```bash
npm ls pino pg --depth=0
```

Upload and start as in steps 2 and 3. Then, on the server, **before** running `node server.js`, install them into the standalone folder with the same versions:

```bash
cd ~/mango-shop/.next/standalone
npm install --no-save pino@X.X.X pg@X.X.X
```

Replace `X.X.X` with the versions `npm ls` showed. If the project doesn't list `pino` directly, `npm ls` shows it nested under another package, such as `payload`. Use that version.

Both are plain JavaScript packages, so installing them on Linux works without any Windows/Linux mismatch.

Either way, if errors appear after starting, paste the first 10–15 lines.

npm ls pino pg --depth=0

npm build EAGAIN spawn error
19h ago

towfi@Towfique UCRT64 ~/Documents/mango-shop/mango-shop
$ grep -rlE "require\(\"(pino|pg)\"\)|from \"(pino|pg)\"" .next/standalone/.next/server | head
.next/standalone/.next/server/app/(frontend)/[lang]/blog/page.js
.next/standalone/.next/server/app/(frontend)/[lang]/blog/[slug]/page.js
.next/standalone/.next/server/app/(frontend)/[lang]/cart/page.js
.next/standalone/.next/server/app/(frontend)/[lang]/category/[slug]/page.js
.next/standalone/.next/server/app/(frontend)/[lang]/checkout/page.js
.next/standalone/.next/server/app/(frontend)/[lang]/order/[number]/page.js

Show more

The build still loads them from `node_modules`, so you'll add them on the server. `pg` is 8.20.0. `pino` didn't appear because another package depends on it, so look one level deeper:

```bash
npm ls pino
```

You'll see something like `└┬ payload@3.x.x  └── pino@9.x.x`. Note that `pino` version.

Step 1: Pack and upload (on your PC)

```bash
rm -f build.tar.gz
tar -czf build.tar.gz -C .next standalone
scp -P 22 build.tar.gz shimanto1@69.57.172.207:~/mango-shop/
```

Step 2: Unpack on the server

```bash
```

npm build EAGAIN spawn error
19h ago

```
cd ~/mango-shop
pkill -u $USER -f node
rm -rf .next/standalone
tar -xzf build.tar.gz -C .next
cp .env .next/standalone/
cd .next/standalone
```

Step 3: Install the two packages

Replace `9.x.x` with the version from `npm ls pino`:

bash

```
UV_THREADPOOL_SIZE=1 npm install --no-save --no-audit --no-fund pg@8.20.0 pino@9.x.x
```

Check they're there:

bash

```
ls node_modules | grep -E "^pino$|^pg$"
```

Step 4: Start

bash

```
TOKIO_WORKER_THREADS=1 RAYON_NUM_THREADS=1 UV_THREADPOOL_SIZE=1 \
PORT=3000 HOSTNAME=0.0.0.0 node server.js
```

Open the homepage, a product page, and `/admin`. If a new "Cannot find module X" error appears, install `X` the same way as in step 3, using the version from `npm ls X` on your PC. If it's a different kind of error, paste the first 10–15 lines.

Yes, it's running. What next? Is there anything left to do?

npm build EAGAIN spawn error
19h ago

Setting up persistent hosting with auto-restart and domain routing.

Yes, it's live. Four things are left. The first one matters most: right now the site stops as soon as you close the SSH window.

1. Keep the site running after you log out

Stop the running server with `Ctrl+C`, then install PM2, which keeps Node apps running and restarts them if they crash:

bash

```
npm install -g pm2
cd ~/mango-shop/.next/standalone
TOKIO_WORKER_THREADS=1 RAYON_NUM_THREADS=1 UV_THREADPOOL_SIZE=1 \
PORT=3000 HOSTNAME=0.0.0.0 pm2 start server.js --name mango-shop
pm2 save
```

Useful commands:

bash

```
pm2 status                # is it running?
pm2 logs mango-shop       # see errors
pm2 restart mango-shop    # restart
```

If `npm install -g pm2` fails with a thread error, use `UV_THREADPOOL_SIZE=1 npm install -g pm2`.

To start the site again after a server reboot, run `pm2 resurrect`. On shared hosting, `pm2 startup` usually needs root, so if your hosting panel has a **Node.js app** or **startup command** setting, use that instead of PM2 for auto-start.

2. Point your domain to it

In your hosting panel, check that the domain's Node.js app or reverse proxy is set to **port 3000**, or change `PORT=3000` above to the port the panel gives you. Then open the domain in your browser, test the checkout page and `/admin`, and make sure SSL (https) is on.

npm build EAGAIN spawn error
19h ago

3. Clean up on the server

bash

```
rm ~/mango-shop/build.tar.gz
```

The `next.config.ts` edits on the server (`experimental`, `ignoreBuildErrors`) don't matter anymore because the site now runs the version built on your PC. Undoing them keeps things tidy, though.

4. Save a deploy script for future updates

The `pg` / `pino` install has to be repeated every deploy, so save these steps as a script. On the server:

bash

```
nano ~/deploy.sh
```

Paste this, replacing `9.x.x` with your pino version:

```bash
#!/bin/bash
set -e
cd ~/mango-shop
pm2 stop mango-shop || true
rm -rf .next/standalone
tar -xzf build.tar.gz -C .next
cp .env .next/standalone/
cd .next/standalone
UV_THREADPOOL_SIZE=1 npm install --no-save --no-audit --no-fund pg@8.20.0 pino@9.x.x
pm2 restart mango-shop --update-env
rm ~/mango-shop/build.tar.gz
echo "Deployed."
```

npm build EAGAIN spawn error
19h ago

Save it, then make it runnable with `chmod +x ~/deploy.sh`.

From then on, each update is three steps.

On your PC:

```bash
npm run build
tar -czf build.tar.gz -C .next standalone
scp -P 22 build.tar.gz shimanto1@69.57.172.207:~/mango-shop/
```

On the server:

```bash
~/deploy.sh
```

If you add `--webpack` to the build script in your PC's `package.json`, as I suggested earlier, `npm run build` will always produce the right kind of build.

```
npm install -g pm2
```

Where should I run this command? Currently, I'm in .next/standalone

You can run it right there. The `-g` flag installs PM2 globally for your user, under your nvm Node folder, so it doesn't matter which folder you're in.

Since you're already in `.next/standalone`, run everything in order:

npm build EAGAIN spawn error
19h ago

```bash
npm install -g pm2
TOKIO_WORKER_THREADS=1 RAYON_NUM_THREADS=1 UV_THREADPOOL_SIZE=1 \
PORT=3000 HOSTNAME=0.0.0.0 pm2 start server.js --name mango-shop
pm2 save
pm2 status
```

Stop the old `node server.js` with `Ctrl+C` first, if it's still running, so the two don't fight over port 3000.

`pm2 status` should show `mango-shop` as **online**. After that you can close the SSH window and the site will keep running.

One thing to know: PM2 is tied to the Node version that was active when you installed it (v24 via nvm). If you switch Node versions later with `nvm use`, you'll need to reinstall PM2.

Site is now running, but I can't create user

Diagnosing a Payload CMS admin setup failure.

To find the cause, I need the actual error. "Can't create user" can come from several different problems. Run this on the server, then try creating the user again in the browser:

```bash
pm2 logs mango-shop --lines 50
```

Paste the new lines that appear when you click create. If the browser shows a message on the form, such as a red toast, tell me what it says too.

Here are the most likely causes, so you can check while you're there.

1. The database tables don't exist yet (most common)

npm build EAGAIN spawn error
19h ago

In production, Payload with PostgreSQL doesn't create tables automatically. You have to run migrations. The logs would show errors like `relation "users" does not exist`. To fix it, run this on the server from the full project folder, not the standalone one:

```bash
cd ~/mango-shop
ls src/migrations          # check that migration files exist
UV_THREADPOOL_SIZE=1 TOKIO_WORKER_THREADS=1 RAYON_NUM_THREADS=1 npx payload migrate
pm2 restart mango-shop
```

2. The server URL in `.env` doesn't match your domain

If `NEXT_PUBLIC_SERVER_URL`, `PAYLOAD_PUBLIC_SERVER_URL`, or `SERVER_URL` still says `http://localhost:3000` while you're visiting `https://yourdomain.com`, the login cookie gets rejected and creating a user fails silently. Check with:

```bash
grep -iE "SERVER_URL|PAYLOAD_SECRET|DATABASE" ~/mango-shop/.env
```

Don't paste the secret values here, only the variable names and the URL. If the URL is wrong, fix it in **both** `~/mango-shop/.env` and `.next/standalone/.env`. `NEXT_PUBLIC_` values are baked into the build, so you'd also need to rebuild on your PC with the corrected `.env` and redeploy.

3. You're using http instead of https

If the site is on plain `http://`, secure cookies won't be saved. Turn on SSL for the domain in your hosting panel and use the `https://` address.

The logs will show which of these it is, or whether it's something else.

npm build EAGAIN spawn error
19h ago

shimanto1@s12716:~/mango-shop/.next/standalone$ pm2 logs mango-shop
[TAILING] Tailing last 15 lines for [mango-shop] process (change the value with --lines option)
/var/www/30d5dc46-92a9-4087-a094-c8e6ee826b10/.pm2/logs/mango-shop-error.log last 15 lines:
/var/www/30d5dc46-92a9-4087-a094-c8e6ee826b10/.pm2/logs/mango-shop-out.log last 15 lines:
0|mango-sh | ▲ Next.js 16.4.3
0|mango-sh | - Local: http://localhost:3000

Show more

Debugging the first-user creation restriction error.

The logs show the database is working, since there are no "relation does not exist" errors. The message `You are not allowed to perform this action` means Payload **received** the request but **refused** it on permission grounds. Which fix you need depends on where you're creating the user.

First, check whether any user already exists

Run on the server:

```bash
curl -s http://localhost:3000/api/users/init
```

- `{"initialized":true}` means an admin user **already exists**, probably from an earlier run. Payload only lets you create the "first user" when there are none, which explains the refusal. Go to **A**.
- `{"initialized":false}` means no users exist, so the problem is the login cookie. Go to **B**.

A. A user already exists: log in or reset the password

Go to `https://yourdomain.com/admin/login` and log in. If you don't know the password, click **Forgot password** and enter the admin email. Your site has no email service set up (that's the `No email adapter` warning), so the reset email is printed into the logs instead of sent:

npm build EAGAIN spawn error
19h ago

```bash
pm2 logs mango-shop --lines 80
```

Find the link containing `reset-password?token=...`, open it in your browser, and set a new password. If the link starts with `http://localhost:3000`, replace that part with your domain.

To see which email the admin account uses:

```bash
cd ~/mango-shop
grep -iE "DATABASE" .env | cut -d= -f1      # only shows the variable name
```

Tell me the database type (Postgres URL or another) and I'll give you a one-line command to list the user emails.

B. No users exist, or you're logged in but still refused

The session cookie isn't reaching the server. Check these:

1. **Use https.** Open the site with `https://`, not `http://`. If SSL isn't on, enable it in your hosting panel.
2. **Check the server URL in `.env`:**

```bash
```

```
grep -iE "SERVER_URL" ~/mango-shop/.next/standalone/.env
```

It must be exactly your live address, for example `https://mangoshop.com` with no trailing slash, and not `localhost`.
3. **Clear cookies** for the site in your browser, or try a private window, then try again.

If the URL in `.env` was wrong, fix it on your PC, rebuild, upload, and run `~/deploy.sh`. A rebuild is needed because `NEXT_PUBLIC_` values are baked into the build.

C. Customers can't register on the shop

If "create user" means shoppers signing up on the store rather than the admin panel, the `users` collection may only allow admins to create accounts. Tell me which page you're on, and paste `src/collections/Users.ts` (or `Customers.ts`) from your PC. The fix would be in its `access.create` rule.

Run the `curl` command and tell me what it returns and which page you're creating the user from.

npm build EAGAIN spawn error
19h ago

host metrics | cpu: 71.1% | ram usage: 59.6%
shimanto1@s12716:~/mango-shop/.next/standalone$ pm2 logs mango-shop
[TAILING] Tailing last 15 lines for [mango-shop] process (change the value with --lines option)
/var/www/30d5dc46-92a9-4087-a094-c8e6ee826b10/.pm2/logs/mango-shop-error.log last 15 lines:
/var/www/30d5dc46-92a9-4087-a094-c8e6ee826b10/.pm2/logs/mango-shop-out.log last 15 lines:
0|mango-sh | ▲ Next.js 16.4.3

Show more

Diagnosing why the server response came back empty.

An empty `curl` result usually means the request was redirected or returned an error with no body, so we need to see the status code. Run these two:

```bash
curl -i http://localhost:3000/api/users/init
curl -i https://shimantoambazar.com/api/users/init
```

Paste the first ~10 lines of each, the part starting with `HTTP/1.1 ...`.

Here's how to read the result:

What you see	What it means
`200` and `{"initialized":true}`	An admin already exists. Log in, or use **Forgot password**; the reset link will show up in `pm2 logs`.
`200` and `{"initialized":false}`	No users yet. The create form should work, so we'll check Payload's settings.
`404`	Your admin collection isn't called `users`.
`307` / `308` redirect	The site's middleware is redirecting `/api` requests, which can also break the admin panel.

If you get a `404`, run this on your PC to find the right collection name:

```bash
cd ~/Documents/mango-shop/mango-shop
grep -rn "auth:" src/collections -l
```

Paste the file names, and tell me two more things:

1. **Which page** are you creating the user on? For example `/admin/create-first-user`, the Users list inside the admin panel, or a sign-up page on the shop.
2. **What the browser shows** when it fails, such as a red message on the form.

With those, I can tell you the exact fix.

npm build EAGAIN spawn error
19h ago

shimanto1@s12716:~/mango-shop$ curl -s http://localhost:3000/api/users/init
shimanto1@s12716:~/mango-shop$ grep -iE "SERVER_URL" ~/mango-shop/.next/standalone/.env
NEXT_PUBLIC_SERVER_URL=https://shimantoambazar.com
shimanto1@s12716:~/mango-shop$ curl -i http://localhost:3000/api/users/init
HTTP/1.1 200 OK
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
X-Frame-Options: SAMEORIGIN

Show more

Debugging the first-user registration permission check.

That narrows it down. `{"initialized":false}` means **no users exist**, and the API works through your domain with HTTPS (LiteSpeed is proxying correctly). Payload should allow creating the first user, so something is refusing that specific request. Let's call the first-user endpoint directly to

see the real error, which may also just create the user.

Step 1: Try creating the first user with curl

On the server, replace the email and password with your own:

```bash
curl -i -X POST https://shimantoambazar.com/api/users/first-register \
  -H "Content-Type: application/json" \
  -H "Origin: https://shimantoambazar.com" \
  -d '{"email":"you@example.com","password":"YourStrongPassword123"}'
```

- If you get `200` and JSON containing your email, the user was created. Go to `https://shimantoambazar.com/admin/login` and log in.
- If you get `400` with a message about missing fields (like `name` or `roles`), add them to the `-d` part, for example:
  `-d '{"email":"you@example.com","password":"YourStrongPassword123","name":"Admin","roles":["admin"]}'`
- If you get `403`, paste the full response. That means a custom rule in your code is blocking it, and I'll need to see `src/collections/Users.ts` from your PC.

Step 2: Tell me which page you used

Was it `/admin/create-first-user`, the admin login page, or a sign-up or account page on the shop itself? If it was a shop page, that explains the refusal. That form creates users through the normal `create` route, which your Users collection probably limits to admins. Creating the admin with step 1 should then fix the admin side, and we can look at customer sign-up separately.

Paste the curl response either way.

npm build EAGAIN spawn error
19h ago

{"initialized":false}shimanto1@s12716:~/mango-shop$ curl -i -X POST https://shimantoambazar.cshimanto1@s12716:~/mango-shop$ curl -i -X POST https://shimantoambazar.com/api/users/first-reshimanto1@s12716:~/mango-shop$ curl -i -X POST https://shimantoambazar.com/api/users/first-register \
-H "Content-Type: application/json" \
-H "Origin: https://shimantoambazar.com" \
-d '{"email":"towfique.work@gmail.com","password":"M13919821u@"}'
HTTP/2 403
x-content-type-options: nosniff

Show more